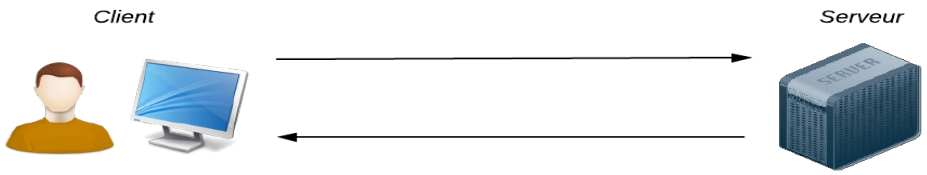
R4.01

websocket

Websocket

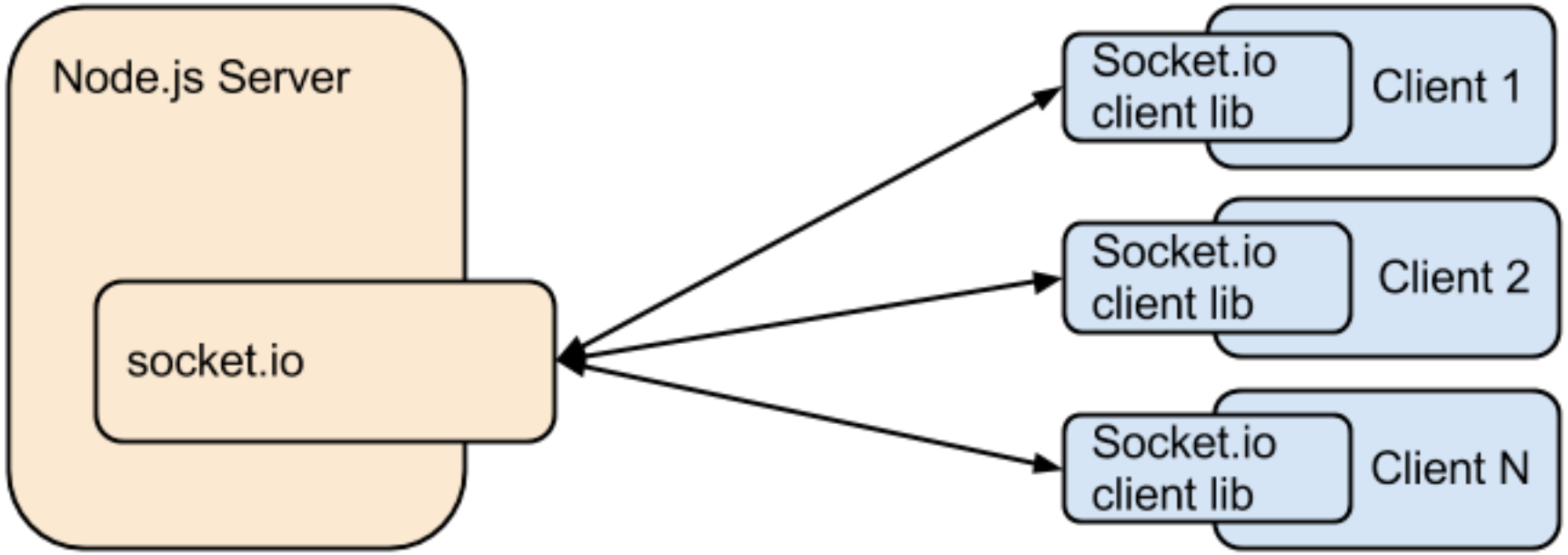
Sans websocket (client – serveur)



Avec websocket



Socket.io



Rem : socket. Io n’est pas une implémentation des web socket (RFC 6455), il ne respecte pas l’intégralité du standard

Servir des pages statiques

- ```
  "use strict"
  import express from 'express'
  import http from 'http'
  //recupération de l'url absolue du module
  import path from 'path'
  import {fileURLToPath} from 'url';

  const __filename =
  fileURLToPath(import.meta.url); //chemin
  absolu vers le module
  const __dirname = path.dirname(__filename);
  //répertoire du module


  const app = express()
     .use(express.static(path.join(__dirname,
  'public')))
  ```

…

```
const server = http.createServer(app);
server.listen(3000, () => {
   console.log("server running on 3000");
})
```
```

Obtenir une connexion

- Serveur
-
```
import { Server } from "socket.io";
const io = new Server(server, {
    cors: {
        origin: 'http://localhost:3000',
        methods:['GET','POST']
    }
})

io.on('connection',

(socket)=>console.log("connecté"))
;
```

- Client

```
<script
src="/socket.io/socket.io.js"></script
>


const socket = io();
```

Socket.io

Emission

- Sur une socket

socket.emit(**'an-event'**,data1, …);

- Sur toutes les sockets

io.emit(**'an-event'**,data1, …);

- Sur toues les sockets sauf celle d'origine

socket.broadcast.emit **'an-event'**,data1, …);

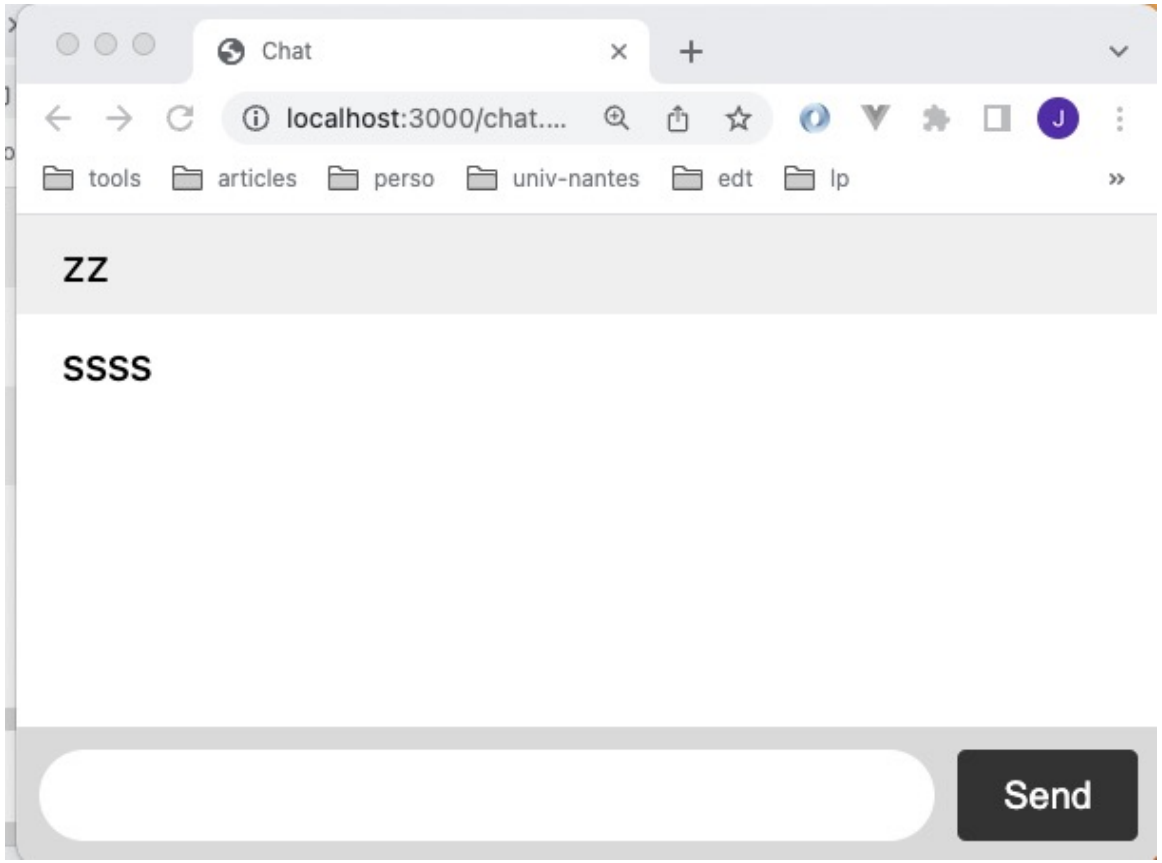
Réception

socket.on(**'an-event', function(data){**

})

Rem : d'autres événements et d'autres flag (socket.io)

Exemple un chat



Exemple un chat : événements

Serveur	Client
Pas d’état	L’état est dans la vue (li)
Sur réception du message « chat message » broadcast	Envoie du message (« chat message ») après submit

Les espaces de nommage

Idée : partager une socket en la partitionnant en namespace

Serveur

```
const nsp = io.of('/my-namespace');
nsp.on('connection',
function(socket){
console.log('someone connected');
});
nsp.emit('hi', 'everyone!');
```

Client

```
const socket = io('/my-namespace');
```

Les rooms

Idée partitionner un namespace, possibilité de joindre ou quitter une room

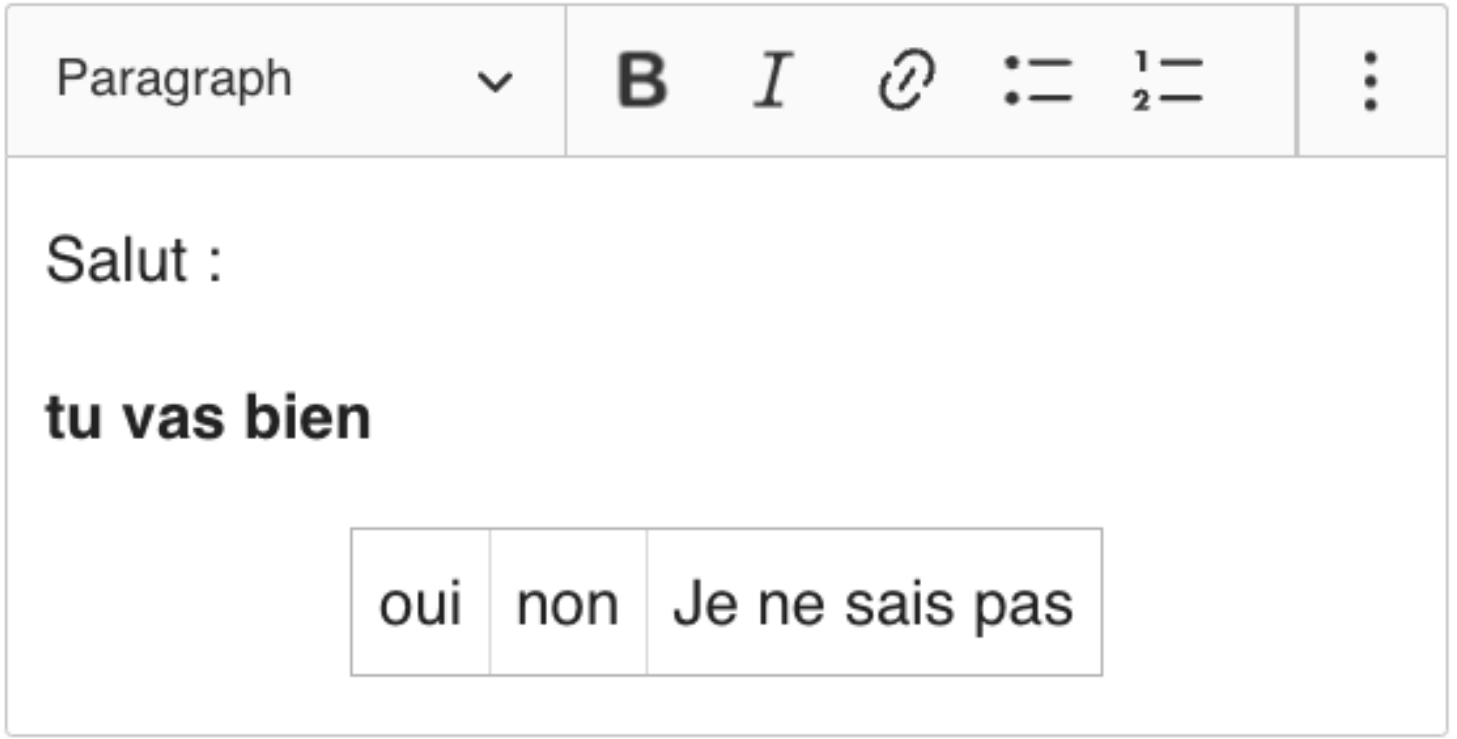
Serveur

```
io.on('connection', function(socket){
socket.join('some room');
});
```

```
io.to('some room').emit('some event’);
```

…

Editeur partagé



Idées

- Modèle : string
- Serveur
  - à la connexion émission event1 transportera le modèle
  - Ecoute sur event2
    - Mets à jour son modèle
    - On émet en broadcast le modèle
- Client
  - A la connexion on écoute l'event1 pour mettre à jour le modèle de l'éditeur
  - A chaque modification, il émet sur l'event2 vers le serveur le modèle